

Auto Session Handoff for Claude Code

A per-project handoff system that captures where you left off — even after a rate-limit — so a fresh session can resume without replaying the whole conversation.

1. The problem this solves

When a Claude Code session stops mid-work — whether from an intentional exit, a hard kill, or an API rate limit — you have three options for coming back:

- **--continue**: replays the entire prior transcript. After the 1-hour prompt-cache TTL expires, that replay is billed at full input-token rate. Expensive.
- **Fresh session**: cheap, but I know nothing about what happened before.
- **/context-save + /context-restore** (gstack skills): solves it, but 100% manual — if you forget to save before quitting, the thread is lost.

This system adds **three shell hooks** to Claude Code that automate the checkpointing so the fresh-session route becomes viable by default.

2. Architecture

Three hook scripts wired into `~/.claude/settings.json`. All write to / read from the existing gstack checkpoints directory, so `/context-restore` keeps working unchanged.

Session start ■■ session-start.sh ■■ inject one-line pointer via additionalContext
Turn end ■■ stop-reminder.sh ■■ nudge to run /context-save when gated
Session end ■■ session-end.sh ■■ dump AUTO-*.md handoff from transcript tail

Note: The critical property: **session-end.sh** is **pure bash + jq**. No Claude API call. So it works even when your rate limit is exhausted.

3. Files installed

Three new scripts plus one edited config file:

File	Purpose
<code>~/.claude/hooks/gstack-session/session-end.sh</code>	Reads transcript tail, writes AUTO handoff .md on session end. Pure shell.
<code>~/.claude/hooks/gstack-session/session-start.sh</code>	Finds newest handoff (≤ 14 days old), injects one-line pointer via additionalContext.
<code>~/.claude/hooks/gstack-session/stop-reminder.sh</code>	Gated soft nudge to run /context-save (≥ 15 turns AND ≥ 3 dirty files AND no recent s
<code>~/.claude/settings.json</code>	Wires the three hooks alongside existing entries. Backup at <code>.bak-&lt;epoch&gt;</code> .

4. Where handoffs are stored (per project)

Handoff files live at:

```
$GSTACK_STATE_ROOT/projects/$SLUG/checkpoints/  
■■■ AUTO-YYYYMMDD-HHMMSS-<title-slug>.md    ← written by SessionEnd hook  
■■■ YYYYMMDD-HHMMSS-<title-slug>.md          ← written by manual /context-save
```

How the paths resolve

- **GSTACK_STATE_ROOT**: from ~/.claude/skills/gstack/bin/gstack-paths. Fallback chain: \$GSTACK_HOME → \$CLAUDE_PLUGIN_DATA (if plugin) → \$HOME/.gstack → .gstack. On your machine: ~/.gstack.
- **SLUG**: from ~/.claude/skills/gstack/bin/gstack-slug. Derived from git remote (e.g. owner-repo) or the directory basename if no remote. Sanitized to [a-zA-Z0-9._-]. Cached in ~/.gstack/slug-cache/ for consistency across sessions.

Example — this project

```
Project dir: /Users/madhusiddharthsuthagar/Documents/python/madhusiddharths  
Slug:      madhusiddharths  
Handoff dir: ~/.gstack/projects/madhusiddharths/checkpoints/
```

Every project gets its own checkpoints directory — no cross-project bleed. The SessionStart hook only sees handoffs for the project you're currently in, because SLUG changes with the cwd.

Retention

- **AUTO files**: newest 20 kept per project; older ones pruned by session-end.sh.
- **Manual /context-save files**: never pruned automatically.
- **SessionStart pointer**: skipped if the newest checkpoint is older than 14 days.

5. Commands and workflow

The zero-effort case: run nothing. Hooks handle both ends automatically. In practice you'll want to know about two optional commands.

Moment	Command	Why
Start of session	(nothing)	SessionStart auto-injects a one-line pointer if a fresh handoff exists.
Mid-session, if you see the nudge	/context-save	The nudge only fires with substantial dirty work. Running /context-save writes
Before intentional stop on important work	/context-save	Same reason. Optional but recommended for mid-plan execution.
Just quitting normally	Ctrl+D or /exit	SessionEnd writes an AUTO handoff automatically.
Next session, when the pointer appears	/context-restore	Loads the full handoff into my context. Skip if starting unrelated work.

Note: The only command worth memorizing is /context-restore — and only when you actually want to resume prior work.

6. Cost analysis

Size of an AUTO handoff

Section	Approx. chars
Frontmatter (status, branch, timestamp)	~120
files_modified: list (say 30 dirty files)	~1,200
Last user prompt + title	~300
Recent activity: 20 turns × ~250 chars each	~5,000
Notes section (fixed text)	~250
Total worst case	~7,000 chars ≈ ~1,750 tokens
<i>Typical case</i>	<i>~1,000 tokens</i>

Full resume cost, end to end

Item	Tokens
SessionStart pointer (always, if handoff exists)	~50
/context-restore skill preamble (only when invoked)	~500–1,000
The AUTO handoff file content itself	~300–1,750
Total per resume	~850–2,800 tokens

Compared to the alternative

At current Opus 4.7 pricing (~\$15 per million input tokens):

Approach	Tokens replayed	Cost
Fresh session + /context-restore (this system)	~1,500 (typical)	~\$0.02
--continue after cache expiry, 50k-token session	~50,000	~\$0.75
--continue after cache expiry, 150k-token session	~150,000	~\$2.25

Savings $\approx$ 30–100 $\times$ per resume compared to --continue after the 1-hour cache TTL expires. On shorter sessions or ones that got auto-compacted, the gap is smaller but still $>10\times$.

7. Recipe: surviving a 3-hour rate limit

The most valuable case. The critical insight:

Note: Keeping the terminal open through a 3-hour wait does **not** keep the prompt cache alive. The cache TTL is 1 hour of inactivity. After that, your next message pays full input price on the whole transcript — same cost as `--continue`. There is no benefit to leaving the terminal sitting idle past the 1-hour mark.

The recipe

- 1. If the stop-reminder nudge appears earlier in the session, run `/context-save` right then.
- 2. If the limit hits, try `/context-save` once (may succeed if partial quota left). If it errors, skip.
- 3. `/exit` cleanly. The AUTO handoff lands regardless — pure shell, no API needed.
- 4. Wait 3 hours.
- 5. Run `claude fresh` (**not** `--continue`). `SessionStart` shows the pointer.
- 6. Run `/context-restore`. Resume.

Why `/context-save` may not work after the limit

`/context-save` asks me to write the summary — that's an API call, and API calls fail under rate limit. The safety net is the AUTO handoff: pure shell, no API, always works on clean exit. Even if you couldn't checkpoint, you still get the last 20 turns + dirty file list.

8. Example AUTO handoff output

Below is an actual file generated during setup testing. It captured a short mock session about refactoring auth middleware to use JWTs.

```
---
status: auto-saved
branch: main
timestamp: 2026-07-24T14:36:49-05:00
files_modified: []
---

## Working on: sounds good, but keep the redis for token blacklisting

### Summary

Auto-captured on session end. Last user prompt:

> sounds good, but keep the redis for token blacklisting

### Recent activity

- **user** (2026-07-24T20:00:01): help me refactor the auth middleware
  to use JWT tokens instead of session cookies
- **claude** (2026-07-24T20:00:05): Sure. First let me read the current
  middleware implementation to understand what we're working with.
- **claude** (2026-07-24T20:00:10): I see the current impl uses
  express-session with a Redis store. For JWT I'd replace lines
  12-40 with jsonwebtoken verify + a custom Bearer parser.
- **user** (2026-07-24T20:00:20): sounds good, but keep the redis
  for token blacklisting
- **claude** (2026-07-24T20:00:25): Good call. I'll wire the blacklist
  check as a second step after signature verify.
```

Notes

This checkpoint was written automatically by the SessionEnd hook.
It is not a Claude-authored summary; the 'Recent activity' section is a raw transcript tail.
For a structured summary, run `/context-save` before ending the session.

What the pointer looks like on next session start

Prior session handoff available (auto-captured): "sounds good, but keep the redis for token blacklisting" – saved 2026-07-24T14:36:49 on branch main. Run `/context-restore` to load the full handoff if resuming this work.

That single line is what SessionStart injects into my context — about 50 tokens.

9. What the AUTO handoff does and doesn't capture

Captured

- Current git branch
- List of uncommitted files (staged + unstaged)
- Timestamp of the session end
- Your last user prompt (first ~200 chars)
- Last 20 conversational turns, each truncated to ~200 chars

NOT captured

- Tool outputs (bash results, file contents, grep, screenshots) — too noisy
- Assistant thinking blocks — filtered as internal
- Diffs of edits — only filenames; git already knows the 'what changed'
- A synthesized summary — no LLM call means no prose, only excerpts
- Anything past the last 20 turns of a long session

Note: For richer capture (Decisions Made, Remaining Work, Notes), run `/context -save` before exit. It costs one Claude call (~2k tokens) but produces a real structured summary.

10. When the stop-reminder fires

The Stop hook nudges you to run `/context-save` only when **all three** conditions hold:

- Transcript has ≥ 15 conversational turns
- Working tree has ≥ 3 uncommitted files (from `git status --porcelain`)
- No checkpoint file (AUTO- or manual) written in the last 30 minutes

If any one fails, the hook exits silently. It only adds a shell exec (~50 ms) per turn — no Claude tokens spent.

The nudge is emitted as a `systemMessage` (advisory) — it does not block me from stopping.

11. Troubleshooting

Symptom	Likely cause / fix
No pointer shows up on session start	Newest checkpoint is >14 days old, or no checkpoints exist yet, or you're in a different project. Check <code>~/.gstack/projects/<slug>/checkpoints/</code> .
No AUTO file after a session	Session was force-killed (Ctrl+C hard, kill -9) — <code>SessionEnd</code> doesn't fire on hard kills. Use <code>/exit</code> or Ctrl+D.
Nudge fires too often / annoying	Loosen thresholds in <code>stop-reminder.sh</code> (increase 15 turns / 3 files / 30 min).
Handoffs from another project leaking in	Shouldn't happen — SLUG is derived from <code>cwd</code> . If it does, clear <code>~/.gstack/slug-cache/</code> .

Symptom	Likely cause / fix
Want to disable temporarily	Restore backup: <code>cp ~/.claude/settings.json.bak-<epoch> ~/.claude/settings.json</code> and restart.
Want to delete all AUTO handoffs	<code>rm ~/.gstack/projects/*/checkpoints/AUTO-*.md</code>

12. One-page cheat sheet

Action	You do
Normal session end	Ctrl+D or /exit — AUTO handoff written automatically
Mid-plan checkpoint	/context-save (Claude-authored summary, richer than AUTO)
See the reminder nudge	Run /context-save if you care about a clean resume
Start a new session (resuming)	Read the pointer, then /context-restore
Start a new session (unrelated work)	Ignore the pointer
Recover after a rate limit	/exit → wait → fresh claude → /context-restore
List all handoffs for this project	<code>ls ~/.gstack/projects/madhusiddharths/checkpoints/</code>

Generated by Claude Code — Auto Session Handoff hooks installed 2026-07-24.